

Assignment 2

2026-06-06

```
library(caret)
```

```
## Loading required package: ggplot2
```

```
## Loading required package: lattice
```

```
library(class)
library(gmodels)
```

```
mydata <- read.csv("~/Desktop/UniversalBank-1.csv")
```

```
# make dummy variables for Education
```

```
mydata$Education_1 <- ifelse(mydata$Education == 1, 1, 0)
```

```
mydata$Education_2 <- ifelse(mydata$Education == 2, 1, 0)
```

```
mydata$Education_3 <- ifelse(mydata$Education == 3, 1, 0)
```

```
# remove ID, ZIP Code, and original Education
```

```
mydata <- mydata[, -c(1, 5, 8)]
```

```
# normalize data
```

```
norm_model <- preProcess(mydata, method = c("range"))
```

```
UniversalBank_normalized <- predict(norm_model, mydata)
```

```
# check data
```

```
names(UniversalBank_normalized)
```

```
## [1] "Age"           "Experience"     "Income"
## [4] "Family"        "CCAvg"         "Mortgage"
## [7] "Personal.Loan" "Securities.Account" "CD.Account"
## [10] "Online"        "CreditCard"    "Education_1"
## [13] "Education_2"   "Education_3"
```

```
table(UniversalBank_normalized$`Personal Loan`)
```

```
## < table of extent 0 >
```

```
names(UniversalBank_normalized)
```

```
## [1] "Age"           "Experience"     "Income"
## [4] "Family"        "CCAvg"         "Mortgage"
## [7] "Personal.Loan" "Securities.Account" "CD.Account"
## [10] "Online"        "CreditCard"    "Education_1"
## [13] "Education_2"   "Education_3"
```

```
# split into training and validation
set.seed(123)
```

```
Index_Train <- createDataPartition(
  UniversalBank_normalized$Personal.Loan,
  p = 0.6,
  list = FALSE
)
```

```
Train <- UniversalBank_normalized[Index_Train, ]
Validation <- UniversalBank_normalized[-Index_Train, ]
```

```
# create predictor datasets
```

```
Train_Predictors <- Train[, names(Train) != "Personal.Loan"]
Validation_Predictors <- Validation[, names(Validation) != "Personal.Loan"]
```

```
# create labels
```

```
Train_labels <- Train$Personal.Loan
Validation_labels <- Validation$Personal.Loan
```

```
Predicted_Validation_labels <- knn(
  Train_Predictors,
  Validation_Predictors,
  cl = Train_labels,
  k = 1
)
```

```
head(Predicted_Validation_labels)
```

```
## [1] 0 0 0 0 0 1
## Levels: 0 1
```

```
new_customer <- data.frame(
  Age = 40,
  Experience = 10,
  Income = 84,
  Family = 2,
  CCAvg = 2,
  Mortgage = 0,
  Securities.Account = 0,
  CD.Account = 0,
  Online = 1,
  CreditCard = 1,
  Education_1 = 0,
  Education_2 = 1,
  Education_3 = 0,
  check.names = FALSE
)
```

```
new_customer <- new_customer[, names(Train_Predictors)]
```

```
knn_customer_k1 <- knn(
  Train_Predictors,
  new_customer,
  cl = Train_labels,
  k = 1
)

knn_customer_k1
```

```
## [1] 1
## Levels: 0 1
```

Answer to Question 1

The customer was classified as 1, indicating that the customer is predicted to accept the personal loan offer.

```
Train$Personal.Loan <- as.factor(Train$Personal.Loan)
Validation$Personal.Loan <- as.factor(Validation$Personal.Loan)
```

```
set.seed(123)

Search_grid <- expand.grid(k = c(1, 3, 5, 7, 9, 11, 13, 15))

model <- train(
  Personal.Loan ~ .,
  data = Train,
  method = "knn",
  tuneGrid = Search_grid
)

model
```

```
## k-Nearest Neighbors
##
## 3000 samples
## 13 predictor
## 2 classes: '0', '1'
##
## No pre-processing
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 3000, 3000, 3000, 3000, 3000, 3000, ...
## Resampling results across tuning parameters:
##
## k Accuracy Kappa
## 1 0.9512419 0.6818549
## 3 0.9460467 0.6334523
## 5 0.9454829 0.6127418
## 7 0.9432327 0.5837862
## 9 0.9410138 0.5551846
## 11 0.9389146 0.5298781
## 13 0.9363503 0.5014884
## 15 0.9347180 0.4827402
```

```
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was k = 1.
```

Question 2 Answer

To determine a value of k that balances overfitting and ignoring predictor information, I used the train() function in the caret package to evaluate several odd values of k. Based on the accuracy results, k = 1 produced the highest accuracy and was selected as the optimal value. Therefore, the best choice of k for this dataset is k = 1.

```
Predicted_Validation_labels <- knn(
  Train_Predictors,
  Validation_Predictors,
  cl = Train_labels,
  k = 1
)

CrossTable(
  x = Validation_labels,
  y = Predicted_Validation_labels,
  prop.chisq = FALSE
)
```

```
##
##
##      Cell Contents
## |-----|
## |              N |
## |      N / Row Total |
## |      N / Col Total |
## |      N / Table Total |
## |-----|
##
##
## Total Observations in Table:  2000
##
##
##      | Predicted_Validation_labels
## Validation_labels |          0 |          1 | Row Total |
## -----|-----|-----|-----|
##           0 |      1782 |         16 |      1798 |
##           |      0.991 |      0.009 |      0.899 |
##           |      0.967 |      0.101 |           |
##           |      0.891 |      0.008 |           |
## -----|-----|-----|-----|
##           1 |         60 |        142 |        202 |
##           |      0.297 |      0.703 |      0.101 |
##           |      0.033 |      0.899 |           |
##           |      0.030 |      0.071 |           |
## -----|-----|-----|-----|
##      Column Total |      1842 |        158 |      2000 |
##           |      0.921 |      0.079 |           |
```

```
## -----|-----|-----|-----|
##
##
```

Question 3 Answer

Using the optimal value of $k = 1$, these were the results of the confusion matrix:

True Negatives = 1782

False Positives = 16

False Negatives = 60

True Positives = 142

```
(1782 + 142) / 2000
```

```
## [1] 0.962
```

Accuracy is 96.2%.

Question 4 Answer

Using the optimal value of $k = 1$, the customer was classified as 1, indicating that the customer is predicted to accept the personal loan offer.

```
# repartition the data into training, validation, and test sets
set.seed(123)
```

```
Index_Train <- createDataPartition(
  UniversalBank_normalized$Personal.Loan,
  p = 0.5,
  list = FALSE
)
```

```
Train <- UniversalBank_normalized[Index_Train, ]
Remaining <- UniversalBank_normalized[-Index_Train, ]
```

```
Index_Validation <- createDataPartition(
  Remaining$Personal.Loan,
  p = 0.6,
  list = FALSE
)
```

```
Validation <- Remaining[Index_Validation, ]
Test <- Remaining[-Index_Validation, ]
```

```
# create predictor datasets
Train_Predictors <- Train[, names(Train) != "Personal.Loan"]
Validation_Predictors <- Validation[, names(Validation) != "Personal.Loan"]
Test_Predictors <- Test[, names(Test) != "Personal.Loan"]

# create labels
```

```
Train_labels <- Train$Personal.Loan
Validation_labels <- Validation$Personal.Loan
Test_labels <- Test$Personal.Loan
```

```
Train_Pred <- knn(
  Train_Predictors,
  Train_Predictors,
  cl = Train_labels,
  k = 1
)
```

```
Validation_Pred <- knn(
  Train_Predictors,
  Validation_Predictors,
  cl = Train_labels,
  k = 1
)
```

```
Test_Pred <- knn(
  Train_Predictors,
  Test_Predictors,
  cl = Train_labels,
  k = 1
)
```

```
CrossTable(
  x = Train_labels,
  y = Train_Pred,
  prop.chisq = FALSE
)
```

```
##
##
##      Cell Contents
## |-----|
## |                      N |
## |      N / Row Total |
## |      N / Col Total |
## |      N / Table Total |
## |-----|
##
##
## Total Observations in Table:  2500
##
##
##      | Train_Pred
## Train_labels |      0 |      1 | Row Total |
## -----|-----|-----|-----|
##      0 |    2271 |      0 |    2271 |
##      |    1.000 |    0.000 |    0.908 |
##      |    1.000 |    0.000 |          |
##      |    0.908 |    0.000 |          |
## -----|-----|-----|-----|
```

```
##           1 |           0 |           229 |           229 |
##           |           0.000 |           1.000 |           0.092 |
##           |           0.000 |           1.000 |           |
##           |           0.000 |           0.092 |           |
## -----|-----|-----|-----|
## Column Total |           2271 |           229 |           2500 |
##           |           0.908 |           0.092 |           |
## -----|-----|-----|-----|
##
##
```

```
CrossTable(
  x = Validation_labels,
  y = Validation_Pred,
  prop.chisq = FALSE
)
```

```
##
##
##   Cell Contents
## |-----|
## |               N |
## |   N / Row Total |
## |   N / Col Total |
## |   N / Table Total |
## |-----|
##
##
## Total Observations in Table:  1500
##
##
##           | Validation_Pred
## Validation_labels |           0 |           1 | Row Total |
## -----|-----|-----|-----|
##           0 |           1341 |           16 |           1357 |
##           |           0.988 |           0.012 |           0.905 |
##           |           0.965 |           0.144 |           |
##           |           0.894 |           0.011 |           |
## -----|-----|-----|-----|
##           1 |           48 |           95 |           143 |
##           |           0.336 |           0.664 |           0.095 |
##           |           0.035 |           0.856 |           |
##           |           0.032 |           0.063 |           |
## -----|-----|-----|-----|
##   Column Total |           1389 |           111 |           1500 |
##           |           0.926 |           0.074 |           |
## -----|-----|-----|-----|
##
##
```

```
CrossTable(
  x = Test_labels,
  y = Test_Pred,
```

```
prop.chisq = FALSE
)
```

```
##
##
##      Cell Contents
## |-----|
## |                      N |
## |          N / Row Total |
## |          N / Col Total |
## |          N / Table Total |
## |-----|
##
##
## Total Observations in Table:  1000
##
##
##      | Test_Pred
## Test_labels |      0 |      1 | Row Total |
## -----|-----|-----|-----|
##           0 |      883 |      9 |      892 |
##           |      0.990 |      0.010 |      0.892 |
##           |      0.966 |      0.105 |      0.892 |
##           |      0.883 |      0.009 |      0.892 |
## -----|-----|-----|-----|
##           1 |      31 |      77 |      108 |
##           |      0.287 |      0.713 |      0.108 |
##           |      0.034 |      0.895 |      0.108 |
##           |      0.031 |      0.077 |      0.108 |
## -----|-----|-----|-----|
## Column Total |      914 |      86 |      1000 |
##           |      0.914 |      0.086 |      0.108 |
## -----|-----|-----|-----|
##
##
```

Training Set Accuracy

```
(2271 + 229) / 2500
```

```
## [1] 1
```

Accuracy = 100%

Validation Set Accuracy

```
(1341 + 95) / 1500
```

```
## [1] 0.9573333
```

Accuracy = 95.73%

Test Set Accuracy

```
(883 + 77) / 1000
```

```
## [1] 0.96
```

Accuracy = 96%

Question 5 Answer

Using $k = 1$, the training set achieved an accuracy of 100.0%, correctly classifying all 2,500 observations. The validation set achieved an accuracy of 95.73%, while the test set achieved an accuracy of 96%. The training accuracy was noticeably higher than the validation and test accuracies because $k = 1$ tends to memorize the training data, which can lead to overfitting. However, the validation and test accuracies remained high and were very similar to one another, indicating that the model performed consistently on unseen data. Overall, although there is some evidence of overfitting due to the perfect training accuracy, the small difference between the validation and test accuracies suggests that the model still generalizes well.